RAPPORT DE TP

TP5 - Analyse d'une application Android vulnérable

Reverse Engineering & Sécurité Mobile IoT

Introduction

Ce rapport présente l'analyse de sécurité de l'application Android vulnérable InjuredAndroid (v1.0.12)

L'objectif de ce TP est triple :

- Analyser statiquement l'APK pour identifier les vulnérabilités
- Extraire des données sensibles (flags, credentials, clés API)
- Documenter les techniques de bypass d'authentification

2. Préparation de l'environnement

2.1 Outils installés

Outil	Version	Usage
Jadx	1.4.7	Décompilation APK vers Java
adb (Android Debug Bridge)	36.0.0	Communication avec le device/émulateur
Google Android SDK	36.1.9	Émulateur Android (non fonctionnel - KVM requis)
Frida	Non déployé	Hooking dynamique (Phase 2)
Objection	Non déployé	Exploration dynamique (Phase 3)

2.2 Téléchargement de la cible

L'APK InjuredAndroid a été téléchargée depuis le dépôt GitHub officiel :

```
wget https://github.com/B3nac/InjuredAndroid/releases/download/v1.0.12/InjuredAndroid-1.0.12-release.apk
```

Taille du fichier : 23.58 MB - SHA256 vérifiée.

2.3 Décompilation avec jadx

```
./jadx/bin/jadx -d injured_out/ InjuredAndroid-1.0.12-release.apk
```

Résultat : décompilation avec 17 erreurs mineures (classes obfusquées tierces). Les sources applicatives de b3nac.injuredandroid sont entièrement lisibles.

3. Phase 1 - Analyse statique

3.1 Analyse du AndroidManifest.xml

La lecture du manifest révèle plusieurs composants exportés sans permission de protection, constituant des vecteurs d'attaque directs :

Composant	Type	Risque
FlagEighteenActivity	Activity	Accessible directement sans authentification
ExportedProtectedIntent	Activity	Vulnérable à l'Intent Redirection
QXV0aA (= Auth en Base64)	Activity	Authentification Firebase anonyme exposée
b25lActivity (= oneActivity)	Activity	Activité obfusquée exportée
FlagFiveReceiver	BroadcastReceiver	Reçoit des intents externes sans restriction
TestBroadcastReceiver	Activity	Exposée sans protection

Commandes utilisées :

```
cat injured_out/resources/AndroidManifest.xml | grep -A3 'activity'
grep -B2 'exported.*true' injured_out/resources/AndroidManifest.xml
```

3.2 Recherche de secrets hardcodés

La commande suivante a été exécutée pour identifier les credentials et clés hardcodés :

```
grep -r 'api_key|password|secret|token' injured_out/sources/
```

Résultats notables :

- Référence à google_api_key dans les sources Firebase
- Clé AWS_SECRET déclarée dans strings.xml (vide dans cette version)
- Tokens OAuth et refresh tokens dans les classes d'authentification Google

3.3 Flags découverts par analyse statique

Flag 1 - Credential hardcodé en clair

Fichier : FlagOneLoginActivity.java

La méthode submitFlag() compare directement l'input utilisateur avec une chaîne littérale :

```
if (d.s.d.g.a(editText.getText().toString(), "F1ag_0n3")) {
```

Flag 2 - Activité exportée sans protection

Fichier : FlagTwoActivity.java

Cette activité est marquée exported=true dans le manifest sans aucune permission requise. Le flag est validé par le simple fait d'accéder à l'activité directement via adb :

```
adb shell am start -n b3nac.injuredandroid/.FlagTwoActivity
```

Flag 3 - Secret dans les ressources XML (Base64)

Fichier : FlagThreeActivity.java + strings.xml

La méthode submitFlag() référence une string dont le nom est encodé en Base64 :

```
getString(R.string.cmVzb3VyY2VzX3lv)
```

Décodage du nom : cmVzb3VyY2VzX3lv => resources_yo

Valeur dans strings.xml :

```
<string name="cmVzb3VyY2VzX3lv">F1ag_thr33</string>
```

Flag 4 - Clé encodée Base64 dans une classe Java

Fichier : FlagFourActivity.java + g.java

La vérification utilise une classe dédiée qui décode un tableau de bytes :

```
byte[] a2 = new g().a();
// Dans g.java : private byte[] f1468a =
Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);
```

Décodage : echo "NF9vdmVyZG9uZV9vbWVsZXRz" | base64 -d

3.4 Vulnérabilité : Intent Redirection (ExportedProtectedIntent)

Fichier : ExportedProtectedIntent.java

Cette activité exportée reçoit un Intent et lance l'activité cible sans vérifier la signature de l'appelant :

```
if
(intent2.resolveActivity(getPackageManager()).getPackageName().equals("b3nac
.injuredandroid")) {
```

La protection ne vérifie que le packageName (facilement usurpable) et non la signature de l'application. Exploit via adb :

```
adb shell am start -n b3nac.injuredandroid/.ExportedProtectedIntent \
  --es access_protected_component b3nac.injuredandroid.FlagOneSuccess
```

3.5 Vulnérabilité : Authentification Firebase anonyme exposée (QXV0aA)

L'activité QXV0aA (décodé : Auth) est exportée et permet une authentification anonyme Firebase sans aucune vérification d'identité :

```
firebaseAuth.d().b(this, new a()); // signInAnonymously()
```

N'importe quelle application externe peut déclencher cette authentification anonyme, obtenant potentiellement des tokens d'accès Firebase.

4. Phase 2 - Analyse dynamique avec Frida (théorique)

4.1 Configuration de l'environnement Frida

Installation et démarrage du frida-server sur l'émulateur :

```
adb push frida-server-16.x.x-android-x86 /data/local/tmp/frida-server
adb shell 'chmod 755 /data/local/tmp/frida-server'
adb shell '/data/local/tmp/frida-server &'
```

4.2 Hook de la méthode submitFlag (Flag 1)

Script Frida pour intercepter et bypasser la vérification du Flag 1 :

```
Java.perform(function() {
  var FlagOneActivity =
Java.use('b3nac.injuredandroid.FlagOneLoginActivity');
  FlagOneActivity.submitFlag.implementation = function(view) {
    console.log('[*] submitFlag hooké');
    // Appel forcé avec le bon flag
    return this.submitFlag.call(this, view);
  };
});
frida -U -l hook_flag.js -f b3nac.injuredandroid --no-pause
```

4.3 Bypass SSL Pinning

Script Frida pour contourner le certificate pinning :

```
Java.perform(function() {
  var TrustManagerImpl =
Java.use('com.android.org.conscrypt.TrustManagerImpl');
  TrustManagerImpl.verifyChain.implementation = function() {
    return arguments[0];
  };
});
```

5. Phase 3 - Bypass avec Objection (théorique)

5.1 Commandes Objection

Exploration de l'application :

```
objection -g b3nac.injuredandroid explore
```

Désactivation des protections :

```
android root disable          # Désactiver la détection root
android sslpinning disable    # Désactiver le certificate pinning
```

Inspection des données locales :

```
file ls /data/data/b3nac.injuredandroid/   # Fichiers de l'app
android preferences get                     # SharedPreferences
android hooking list activities              # Lister les activités
```

Lancement direct d'activités protégées :

```
android intent launch_activity b3nac.injuredandroid.FlagEightLogInActivity
```

6. Synthèse des vulnérabilités

6.1 Tableau récapitulatif des flags

Flag	Valeur	Technique	Difficulté
Flag 1	F1ag_0n3	Hardcoded credential en clair	Triviale
Flag 2	Accès direct	Exported Activity sans permission	Triviale
Flag 3	F1ag_thr33	Base64 dans strings.xml	Facile
Flag 4	4_overdone_omelets	Base64 dans classe Java	Facile

6.2 Tableau des vulnérabilités identifiées

Vulnérabilité	Composant	Gravité	OWASP Mobile
Hardcoded credentials	FlagOneLoginActivity	CRITIQUE	M9 - Reverse Engineering
Exported Activity non protégée	FlagTwoActivity, FlagEighteenActivity	HAUTE	M4 - Insecure Auth
Intent Redirection	ExportedProtectedIntent	HAUTE	M4 - Insecure Auth
Secrets en Base64 non chiffrés	g.java, strings.xml	MOYENNE	M9 - Reverse Engineering
Firebase Auth anonyme exposée	QXV0aA	MOYENNE	M4 - Insecure Auth
BroadcastReceiver exporté	FlagFiveReceiver	MOYENNE	M4 - Insecure Auth
Obfuscation insuffisante	QXV0aA, b25lActivity	FAIBLE	M9 - Reverse Engineering

7. Recommandations

7.1 Ne jamais hardcoder de secrets

Toute valeur sensible (mot de passe, clé API, token) ne doit jamais être stockée en clair dans le code source ou les ressources. Utiliser :

- Android Keystore System pour le stockage des clés cryptographiques
- Variables d'environnement injectées à la compilation via BuildConfig
- Services de gestion de secrets (AWS Secrets Manager, HashiCorp Vault)

7.2 Protéger les composants Android exportés

Tout composant exporté doit être protégé par une permission dédiée :

```
<activity android:name=".SensitiveActivity"
  android:exported="true"
  android:permission="b3nac.injuredandroid.ACCESS_SENSITIVE"/>
```

7.3 Valider les Intents reçus

Pour ExportedProtectedIntent, la validation doit inclure :

- Vérification de la signature de l'application appelante
- Validation stricte des extras de l'Intent
- Utilisation de PendingIntent avec des permissions spécifiques

7.4 Chiffrement des données sensibles

L'encodage Base64 n'est pas du chiffrement. Utiliser AES-256-GCM avec une clé stockée dans l'Android Keystore.

7.5 Obfuscation et ProGuard

Activer ProGuard/R8 en production pour rendre l'analyse statique plus difficile, bien que cela ne remplace pas un chiffrement correct.

8. Conclusion

L'analyse statique de l'application InjuredAndroid a permis d'identifier de nombreuses vulnérabilités critiques représentatives des erreurs courantes dans le développement d'applications Android IoT.

Les vulnérabilités les plus graves sont les credentials hardcodés et les composants exportés sans protection, qui permettent un accès non autorisé sans même exécuter l'application. L'analyse dynamique avec Frida et Objection aurait permis d'aller plus loin en interceptant les appels à l'exécution et en bypassant les protections runtime.

Contrainte technique : L'absence de support KVM dans l'environnement VMware Workstation a empêché le lancement de l'émulateur Android x86_64, limitant le TP à la Phase 1. Les Phases 2 (Frida) et 3 (Objection) ont été documentées théoriquement.